

CAPSLOCK
Security gateway for Kubernetes, DevOps and Pipelines

25-26J-043

Project Proposal Report

S A L Guruge

B.Sc. (Hons) Degree in Information Technology Specialising in Cyber
Security

Department of Computer Systems and Engineering

Sri Lanka Institute of Information Technology
Sri Lanka

August 2025

Declaration

We declare that this is our own work, and this proposal does not incorporate without acknowledgement any material previously submitted for a degree or diploma in any other university or Institute of higher learning and to the best of our knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Signature:

Date:

Signature of the Supervisor:



Date: 29/08/2025

Signature of the Co-Supervisor



Date: 29/08/2025

ABSTRACT

Kubernetes is the leading platform to deploy and manage containerized applications today due to its scalability and automation. Workloads in dynamic Kubernetes environments are not yet fully secure, specially for detecting malware. Traditional malware scanners like ClamAV and ICAP are generally deployed manually or at the network edge making them inefficient, difficult to manage, and unable to inspect all intra-cluster traffic. At the same time, the default Horizontal Pod Autoscaler (HPA) of Kubernetes scales workloads only by CPU and memory metrics which do not represent security service health where latency, error rates, and signature freshness have a direct impact on the reliability of detection.

This research proposes the construction and deployment of a Kubernetes-native ICAP Operator that will manage the lifecycle of ICAP and ClamAV services, introduce dynamic health scoring through security-critical metrics, and enable adaptive autoscaling through HPA or KEDA. The system will be tested in a controlled testbed with emulation of workloads and malware traffic, with outputs being compared with baseline deployments utilizing manual configuration and default autoscaling. The expected outcome is increased malware detection accuracy, faster scaling responsiveness and improved resource utilization.

Through the integration of container security and Kubernetes automation frameworks, this research delivers an operationally scalable solution for malware defense in cloud-native deployments.

TABLE OF CONTENT

Declaration	Error! Bookmark not defined.
Abstract	ii
Table Of Content	iii
List Of Figures	v
List Of Tables	vi
List Of Appendices.....	vii
List Of Abbreviations	viii
1. Introduction	1
1.1 Background & Literature Survey	1
1.2 Research Gap	4
1.3 Research Problem.....	6
2. Objectives.....	8
2.1 Main Objectives	8
2.2 Specific Objectives.....	8
3. Methodology	9
3.1 System Diagram	9
3.2 Workload Allocation	11
3.2.1 Work Breakdown Structure	11
3.2.2 Gantt Chart	12
3.2.1 Commercialization	12
4. Project Requirements	13
4.1 Functional Requirements	13
4.2 Non-Functional Requirements	13
4.3 User Requirements	13

6. Budget And Buget Justification	14
7. References	15
8. Appendices	18

LIST OF FIGURES

Figure 1: System Diagram of the ICAP Operator 9

Figure 2: Work Breakdown 11

Figure 3: Gantt Chart 12

LIST OF TABLES

Table 1:Comparison of Existing Methods and Proposed Method	4
Table 2:Technologies and techniques used	10
Table 3:Expenses for the proposed system	14

LIST OF APPENDICES

Appendix- A: Plagiarism report	18
--------------------------------------	----

LIST OF ABBREVIATIONS

Abbreviation	Description
AKS	Azure Kubernetes Service
API	Application Programming Interface
CRD	Custom Resource Definition
CPU	Central Processing Unit
GPU	Graphics Processing Unit
HPA	Horizontal Pod Autoscaler
ICAP	Internet Content Adaptation Protocol
IoT	Internet of Things
K8s	Kubernetes
KEDA	Kubernetes Event-driven Autoscaling
MAC	Mandatory Access Control
eBPF	Extended Berkeley Packet Filter

1. INTRODUCTION

1.1 Background & Literature survey

Background

Containers and Kubernetes have changed the way modern cloud-native applications are deployed and scaled. Kubernetes provides automated scheduling, orchestration, and fault tolerance, making it the de facto standard for containerized environments [1]. However, increased utilization of containers introduces new security concerns. All containers share the same host kernel, and vulnerabilities or misconfigurations can expose workloads to privilege escalation and container escapes [2]. Large-scale studies have also proved that a high percentage of publicly available container images contain outdated packages and high-severity vulnerabilities, which is a severe issue in terms of the security posture of containerized environments [3].

Antivirus engines such as ClamAV have been widely used to handle malware threats. ClamAV provides signature-based detection and can be further extended to content scanning using the Internet Content Adaptation Protocol (ICAP). ICAP-based integration, such as through c-icap, allows proxy and gateway servers to intercept traffic and route it to antivirus engines for inspection [4]. Research has shown that ICAP integration enables transparent offloading of scanning workloads, with client being kept protected without local antivirus software [5]. More recent studies show ClamAV's versatility, being applied in IoT environments, Bluetooth proxies, and hardware-accelerated detection pipelines [6], [7]. Despite this ICAP-based scanning remains widely deployed in static or network-edge configurations, work is still to be done to seamlessly incorporate ICAP-based scanning for malware scan invocation into dynamic Kubernetes clusters.

Kubernetes Operators were introduced as an extension mechanism that makes complex application management tasks easier to automate. Operators encode domain-specific operational logic into controllers and Custom Resource Definitions (CRDs), allowing Kubernetes to reduce manual effort [8]. Kubernetes Operators have also been successfully

demonstrated to automate observable frameworks [9], machine learning workflows [10], and stateful services. However, empirical studies indicates that operators are often buggy and problematic to maintain, highlighting the need for solid design when extending them, particularly in mission-critical services such as security [11].

Literature Survey

Recent research has highlighted that the Kubernetes' default scaling mechanisms, primarily the Horizontal Pod Autoscaler (HPA) are limited because they rely on CPU and memory usage, which do not capture application-specific behavior [12]. Studies have thus proposed advanced autoscaling techniques, combining predictive models, machine learning, and fine-tuning to predict workload fluctuation [13][16]. For example, predictive approaches using hybrid models such as LSTM and Prophet improve accuracy in forecasting workload spikes [14]. Other works indicate that the statistical tuning of autoscaling parameters can significantly reduce resource consumption without degrading the performance [15]. Emerging frameworks such as multi-agent autoscalers enhance resilience against adversarial conditions, such as DDoS attacks or cascading failures [16].

In the meantime, container security surveys reveal an evolving threat landscape. Comprehensive studies have mapped container vulnerabilities into taxonomies of exploits, highlighting kernel-sharing issues and misconfigurations as primary risks [17]. Threat modeling methods such as STRIDE have been applied to containers, identifying weaknesses in supply chains, registries, and orchestrators [18]. Some more specific studies have considered detection of container escape attacks via namespace anomalies [19], Mandatory Access Control (MAC) frameworks [20], and provenance-based detection [21]. Further study has demonstrated how kernel exploits and cross-container attacks using extended Berkeley Packet Filter (eBPF) can bypass isolation feature [22], [23]. Security researchers have also studied container runtime security vulnerabilities, proposing hardening solutions such as enabling user namespaces and performing normal patching [24].

Overall, the studies conclude three sure points:

- Malware detection engines like ClamAV, especially in conjunction with ICAP, are highly relevant but not utilized enough in cloud-native environments.
- Kubernetes Operators are a good level of abstraction for automating such security services but must be developed with caution to be resilient.
- Health-aware and adaptive autoscaling, rather than CPU and memory levels, is needed to maintain performance and security promises in Kubernetes clusters.

1.2 Research Gap

Current research highlights the usefulness of both ICAP and Kubernetes Operators, but their integration for security orchestration is largely unexplored. ICAP-based malware scanning has been successfully applied at network gateways [4], with ClamAV adapted for hardware acceleration [5] and IoT defense [6]. However, these methods are static deployments, lacking the automation and elasticity required in Kubernetes environments.

In the meantime, Kubernetes Operators have emerged as a powerful abstraction for lifecycle automation for workloads ranging from observability stacks [1] to machine learning pipelines [3]. Yet, despite their potential, no prior work has developed a Kubernetes-native Operator for ICAP-based malware scanning. Efforts such as Xu et al. [2] also note that existing Operators are too often filled with bugs and design flaws, which highlights the need for reliability for mission-critical use cases like security.

Autoscaling studies also illustrate progress in workload forecasting [14], but such methods have focused primarily on performance optimization and service availability. They have not been applied to security-relevant signals like ICAP scan latency, backlog length, error rates, or ClamAV signature age, factors that directly influence malware detection effectiveness.

Methods Used	Proposed Method
Manual ICAP/ClamAV setups with basic autoscaling	Automated ICAP Operator with health scoring & adaptive scaling

Table 1: Comparison of Existing Methods and Proposed Method

Existing methods depend on manual ICAP and ClamAV deployments with only basic autoscaling based on CPU or memory. These approaches are difficult to manage, not optimized for security workloads, and often fail to scale effectively under changing traffic. The proposed method introduces a Kubernetes-native ICAP Operator that automates deployment, integrates health scoring to track real performance, and enables adaptive scaling to balance security and efficiency.

1.3 Research Problem

Kubernetes has quickly become the most popular platform for orchestrating containerized applications given its capabilities for scalability and automation [1]. However, despite these benefits, there are still significant challenges to integrating strong security controls in a Kubernetes environment. Malware detection tools, such as ICAP or ClamAV, are often deployed manually or at the network edge [4], which is ineffective for dynamic clusters with changing workloads. Manual deployment is inefficient, contributes to operational overhead, and results in uneven coverage across environments, while scanning only at the edge cannot scan all traffic and file transfers taking place inside the cluster.

On the other hand, Kubernetes Operators have been shown to effectively automate the lifecycle of complex workloads including observability systems, machine learning pipelines, and stateful services [3]. However, no prior research has applied such automated lifecycle models to security-oriented services such as ICAP-based malware scanning. Therefore, organizations are still operating with brittle, manual processes to deploy and maintain antivirus scanning in Kubernetes, which means poor overall embedded security.

The second main issue is autoscaling capability. The default Kubernetes Horizontal Pod Autoscaler (HPA) scales based on CPU and memory usage [12]. While they are great for traditional workloads, they do not reliably represent the health of a security workload. Security workload health is often impacted by scan latency, error rates, backlog depth, and freshness of ClamAV signatures. Some or all of these are direct indicators of the reliability of malware detection, but they are all invisible to the autoscaling model in use. As a result, ICAP services are likely to be under-provisioned and lead to scanning delays and missed detections or over-provisioned and lead to a squandering of computing resources and costs [14].

The primary purpose of this research is to design and implement a Kubernetes-native Operator that automates the life cycle management of ICAP and ClamAV services, allows for dynamic health scoring based on security metrics, and implements adaptive scaling.

The goal is that Kubernetes environments will be able to achieve more effective malware protection while utilizing a more efficient use of resources, addressing a significant gap in the existing processes dealing with security of containers.

Therefore, the research problem can be defined as follows:

How can a Kubernetes-native Operator be developed to manage ICAP-based ClamAV services with dynamic health scoring and adaptive scaling, ensuring strong security and efficient resource use in containerized environments?

2. OBJECTIVES

2.1 Main Objectives

To design and implement a **Kubernetes-native Operator** that automates ICAP and ClamAV service management, integrates **dynamic health scoring**, and enables **adaptive scaling** to enhance malware protection and resource efficiency in containerized environments.

2.2 Specific Objectives

- Develop a Custom Resource Definition (CRD) and Operator framework within Kubernetes to automate the deployment, configuration, and lifecycle management of ICAP and ClamAV services.
- Design and implement a dynamic health scoring mechanism that continuously evaluates security workload health using measurable metrics (scan latency, error rates, backlog depth, signature freshness) and integrate it into the Operator.
- Implement adaptive autoscaling policies that use health scores to trigger Horizontal Pod Autoscaler (HPA) or KEDA scaling events, ensuring at least 20% improvement in resource efficiency and response time compared to default CPU/memory-based scaling.
- Evaluate and benchmark the proposed Operator in a controlled Kubernetes testbed with simulated workloads and malware samples, measuring improvements in detection accuracy, scaling responsiveness and resource utilization.

3. METHODOLOGY

3.1 System Diagram

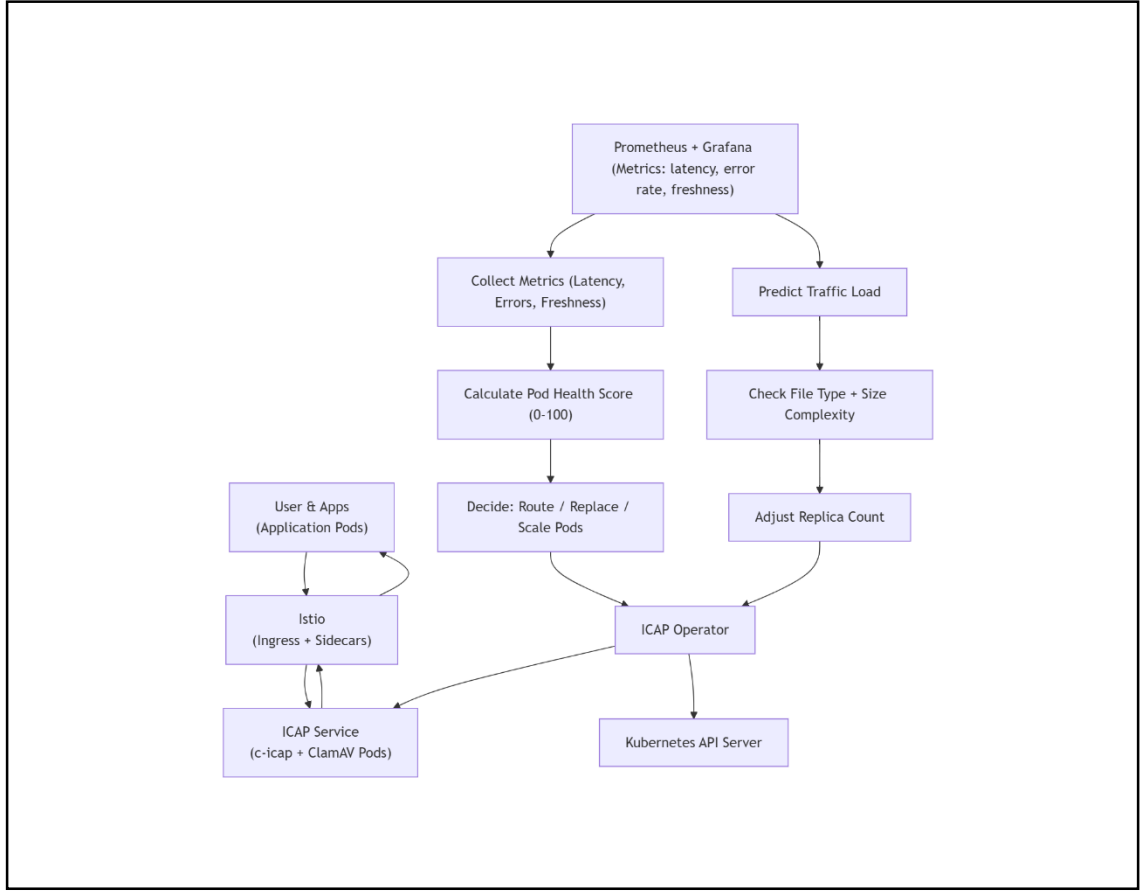


Figure 1: System Diagram of the ICAP Operator

Figure 1 illustrates the overall high-level architectural diagram of the proposed component.

The summary of technologies, techniques and architecture used for is shown in the table (Table 2) below.

Technologies	Kubernetes, ICAP, ClamAV, Prometheus, Grafana
Techniques	Operator pattern, automated lifecycle management, dynamic health scoring, adaptive autoscaling
Architectures	Kubernetes-native Operator architecture, ICAP + ClamAV scanning stack, monitoring and scaling loop

Table 2: Technologies and techniques used

The research will start with setting requirements and success factors for the Kubernetes ICAP Operator including automated life cycle management, health scoring, and adaptive scaling. A test environment will then be set up using local and cloud-based Kubernetes clusters to simulate realistic workloads.

Then the Operator will be designed and developed with a Custom Resource Definition (CRD) to manage ICAP and ClamAV services. Prometheus will be used to collect security metrics, which will be combined into a health score, and this score will drive adaptive scaling using HPA or KEDA.

The system will be validated and evaluated by simulating workloads and malware traffic, with results compared against manual deployments and default autoscaling methods. The evaluation will measure improvements in detection reliability, responsiveness, and resource efficiency.

3.2 Workload Allocation

3.2.1 Work Breakdown Structure

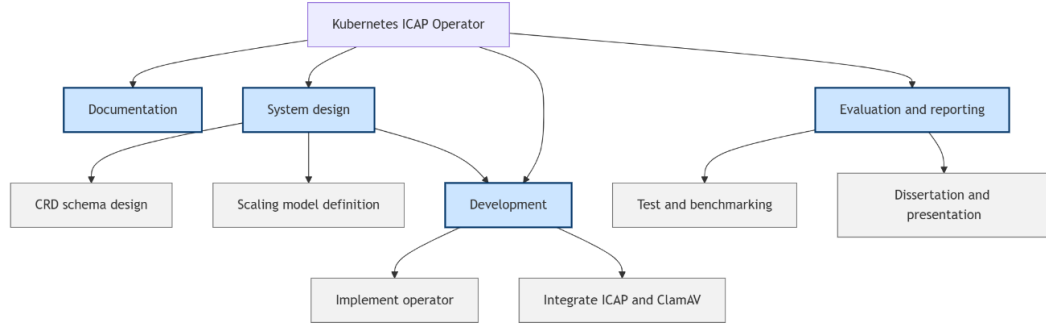


Figure 2: Work Breakdown

The work breakdown for this research component is separated into four major phases, as documentation, system design, development and evaluation.

The documentation phase documents the proposal report preparation at the beginning and final completion of the thesis near the end. The system design involves designing the schema of the Custom Resource Definition (CRD) and identifying the scaling model that will serve as the foundation for future development. Development also involves coding the operator and implementing ICAP with ClamAV to encapsulate service management and malware scanning within Kubernetes. Finally, the operator testing and benchmarking in a Kubernetes platform as well as examination and integration of the outcomes into the thesis and presentation content forms the evaluation and reporting process.

Reporting and documentation are carried out concurrently throughout the project lifecycle.

3.2.2 Gantt Chart

Task	Month														
	Apr	May	Jun	Jul	Aug	Sep	Oct	Nov	Dec	Jan	Feb	Mar	Apr	May	
Brainstorming Sessions															
Team Formation															
Topic Finding															
Background research of the topic															
Topic Assessment															
5.1 Topic Submission & confirmation															
Proposal Evaluation															
6.1 Project Proposal Report															
6.2 Project Proposal Presentation															
Identification of idea I techniques															
Implementation															
8.1 System Design															
8.2 Operator Development															
8.3 Health Scoring and Adaptive Scaling															
8.4 System Integration and Testing															
Final Evaluation															
8.1 Website															
8.2 Log Book															
8.3 Thesis															

Figure 3: Gantt Chart

Figure 3 is a demonstration of how the tasks broken down in Figure 2 will be achieved in terms of time.

3.2.1 Commercialization

The proposed Kubernetes ICAP Operator can be commercialized as a practical security solution for companies using Kubernetes in production. Its ability to automate malware scanning, integrate health scoring, and adaptively scale security workloads make it valuable for enterprises that need strong, efficient security in dynamic environments.

The product can be released in an open-source version for wide adoption, with an enterprise edition offering advanced dashboards, integrations, and support.

Who will use it:

- Cloud service providers running multi-tenant Kubernetes clusters
- Enterprises such as banks, healthcare, and government agencies that must install strong malware protection
- Managed Security Service Providers (MSSPs) which offer Kubernetes security as part of services
- Academia and research testing secure cloud-native stacks

4. PROJECT REQUIREMENTS

4.1 Functional requirements

- The Operator must automatically deploy and manage ICAP and ClamAV pods in Kubernetes.
- It must update ClamAV signatures regularly without interrupting services.
- It must collect and expose metrics such as latency, error rates, backlog depth, and signature freshness.
- It must calculate a dynamic health score based on these metrics.
- It must trigger adaptive scaling of ICAP pods through HPA or KEDA.
- It must provide status updates in the CRD (e.g. ready pods, health score).

4.2 Non-functional requirements

- Scalability: Must handle both small and large-scale Kubernetes clusters.
- Reliability: Services must recover automatically from pod or node failures.
- Performance: Health scores must be updated in near real-time.
- Security: Operator actions must follow Kubernetes RBAC and network policies.
- Usability: CRDs must be easy to configure with minimal YAML complexity.
- Observability: Metrics, logs, and alerts must be Prometheus/Grafana compatible.

4.3 User requirements

- Users must be able to deploy the Operator easily using YAML or Helm charts.
- Users must be able to configure scaling policies and thresholds through CRDs.
- Users must have access to real-time dashboards showing health scores and scaling actions.
- Users must receive alerts when thresholds are crossed (e.g. health score drops).
- Users must be able to test the system using safe malware samples (e.g. EICAR files).
- Users must have documentation and usage guidelines for installation and maintenance.

6. BUDGET AND BUDGET JUSTIFICATION

The implementation of ICAP operator will primarily use open-source tools and student-accessible resources. Table 3 below shows the overall budget of the entire proposed system.

Expenses	
Requirement	Cost (Rs.)
Software & Tools (Kubebuilder)	20,000.00
Total Cost	20,000.00

Table 3: Expenses for the proposed system

7. REFERENCES

- [1] V. Shenoy, A. S. Rathore, and M. K. Jha, “Kubernetes operators for observability: automation in cloud-native monitoring systems,” *Journal of Cloud Computing*, vol. 11, no. 4, pp. 1–13, 2022.
- [2] L. Xu, Z. Liu, and Y. Zhou, “A comprehensive study on bugs in Kubernetes operators,” *Proceedings of the ACM Symposium on Cloud Computing*, pp. 275–288, 2024.
- [3] I. Trapezin and A. Filianin, “Operator-driven lifecycle management for machine learning workflows on Kubernetes,” *International Conference on Cloud Engineering (IC2E)*, pp. 145–154, 2025.
- [4] M. Ilca and S. Lucian, “ICAP protocol integration for SMEs: extending OPNsense with ClamAV malware scanning,” *International Journal of Computer Applications*, vol. 185, no. 23, pp. 34–40, 2023.
- [5] R. Mangipudi, P. Ramamoorthy, and R. Kumar, “Hardware-accelerated ClamAV for real-time malware detection,” *IEEE Access*, vol. 9, pp. 125634–125645, 2021.
- [6] H. Wang and Y. Lai, “Lightweight ICAP malware defense for IoT gateways,” *Future Internet*, vol. 15, no. 2, pp. 45–56, 2023.
- [7] K. N. Karthik, A. Vasan, and M. R. Divakar, “Improving malware detection with GPU-accelerated ClamAV,” *Proceedings of the IEEE International Conference on Cloud Computing Technology and Science (CloudCom)*, pp. 342–349, 2021.
- [8] B. Burns, J. Beda, and B. Grant, *Designing Distributed Systems: Patterns for Kubernetes, Operators, and Beyond*. O’Reilly Media, 2018.
- [9] V. Kumar and R. Patel, “Automating observability frameworks with Kubernetes operators,” *International Journal of Cloud Applications and Computing*, vol. 12, no. 2, pp. 55–67, 2021.
- [10] C. Wu, Z. Li, and L. Chen, “End-to-end ML workflow automation using Kubernetes operators,” *IEEE International Conference on Big Data (BigData)*, pp. 511–520, 2020.
- [11] Y. Xu and W. Song, “A study on reliability and maintainability challenges in Kubernetes operators,” *Journal of Systems and Software*, vol. 200, p. 111656, 2023.

- [12] M. Molleti, “Analysis of Kubernetes Horizontal Pod Autoscaler and limitations in security workloads,” *International Journal of Computer Science Trends and Technology (IJCSST)*, vol. 10, no. 4, pp. 17–23, 2022.
- [13] T. R. Augustyn, F. De Souza, and R. Martins, “Statistical tuning of Kubernetes HPA for resource-efficient scaling,” *IEEE Transactions on Cloud Computing*, pp. 1–12, 2024.
- [14] P. B. Guruge and Y. H. P. P. Priyadarshana, “Time series forecasting-based Kubernetes autoscaling using Facebook Prophet and LSTM,” *Frontiers in Computer Science*, vol. 7, Article 1509165, 2025.
- [15] A. K. Sharma and M. Verma, “Workload-aware autoscaling of microservices using statistical modeling,” *ACM Symposium on Cloud Computing*, pp. 121–132, 2023.
- [16] S. Lee and J. Kim, “Resilient multi-agent autoscaling framework for Kubernetes under adversarial conditions,” *Future Generation Computer Systems*, vol. 152, pp. 15–28, 2024.
- [17] P. Peinl and S. Holzinger, “Security of Docker and Kubernetes: taxonomy of vulnerabilities,” *Journal of Cloud Computing: Advances, Systems and Applications*, vol. 9, no. 1, pp. 1–15, 2020.
- [18] A. Singh, V. Kumar, and P. Gupta, “Applying STRIDE threat modeling to containerized environments,” *International Conference on Information Systems Security and Privacy (ICISSP)*, pp. 233–241, 2021.
- [19] H. Sun and Y. Zhao, “Detecting container escape attacks via namespace anomaly detection,” *IEEE International Conference on Communications (ICC)*, pp. 4554–4560, 2021.
- [20] J. Hwang and K. Park, “Enhancing container isolation with MAC frameworks: SELinux and AppArmor in Kubernetes,” *International Journal of Information Security*, vol. 22, pp. 411–426, 2023.
- [21] F. Yang, L. Zhou, and Z. Zhang, “Provenance-based detection of malicious activities in container environments,” *Computers & Security*, vol. 118, p. 102737, 2022.
- [22] M. H. Kang, D. Lee, and Y. Choi, “Bypassing container isolation with eBPF: threats and defenses,” *IEEE Security & Privacy Workshops (SPW)*, pp. 124–130, 2022.

[23] D. Kim and H. Cho, “Cross-container kernel exploits in Kubernetes: attack and mitigation strategies,” *Journal of Information Security and Applications*, vol. 70, p. 103423, 2023.

[24] A. Morabito and J. Kim, “Container runtime security: vulnerabilities and hardening strategies,” *ACM Computing Surveys*, vol. 56, no. 4, pp. 1–36, 2024.

8. APPENDICES

Appendix A - Plagiarism check